

总体结论

本地开发 Agent 确实比纯对话工具更适合扮演“外部持久记忆系统”角色。它们可以直接访问项目文件、Git 仓库和终端工具，将规则、项目状态、任务和记忆等以文件形式存储和管理。以 OpenAI 的 Codex CLI 为例，它在启动时会读取项目目录（及更上层目录）中的 `AGENTS.md` 文件，将全局指导与项目级指令层叠起来【3†L19-L27】¹。Anthropic 的 Claude Code 也提供了同类功能：它在每次会话开始时读取根目录的 `CLAUDE.md` 文件，并自动构建本地的记忆数据库（保存在 `~/.claude/projects/<project>/memory/`）¹²。相比 ChatGPT 网页端等纯对话接口，这些本地 coding agent 可直接读写文件、运行测试命令并提交代码，使得“长期记忆”更易版本化、审计化。

Codex 能力边界

- **规则读取：** Codex CLI 会在工作前自动加载多个级别的规则文件，包括用户主目录下的 `~/.codex/AGENTS.md`（全局指令）和当前项目及其父目录中的 `AGENTS.override.md`、`AGENTS.md` 及几个备选文件【3†L19-L27】。最多读取每个目录中第一个非空的规则文件，并自上而下合并指令内容。这保证了它可以统一全局和项目的指导原则【3†L19-L27】³。
- **文件修改：** Codex 通过内置的 `apply_patch` 功能来编辑文件：模型输出一个特定的 patch 格式字符串，CLI 解析后直接修改文件系统⁴⁵。它可以对任何仓库中的文件（包括所谓的 memory 文件）进行补丁更新。Codex CLI 会自动将每个补丁作为一次 Git 提交，因此用户无需手动 `git commit`⁵。
- **diff / patch / commit：** 在对文件作出修改后，Codex 支持展示 `git diff` 供用户复查（使用 `/diff` 命令⁶）。修改文件时，它生成补丁自动应用并提交，本地生成的变更日志可通过 `/diff` 或 `/review` 等命令查看⁶⁵。目前 Codex CLI 不会自动更新已有的远程 PR；用户通常需要手动拉取或创建新的 PR 才能将改动推送到远程⁷⁸。
- **适合维护的记忆文件：** 由于 Codex 可以读写任意文件，它理论上可以将项目中的长期记忆文件（如 `AGENTS.md`、自定义的 `memory-ledger`、`project-state.md`、`tasks/` 目录下文件等）当作上下文使用，并通过补丁更新这些文件。不过，目前 Codex 本身并不内置记忆管理逻辑，更多是按需应用补丁；项目中哪些文件作为“记忆”完全取决于用户或团队的约定。一般建议将持续性的、需要版本控制的规则与状态文档放入 Git 仓库，由 Codex 按需读取和编辑，然后生成补丁让用户审查再提交。

Claude Code 能力边界

- **CLAUDE.md：** Claude Code 支持在项目根目录添加 `CLAUDE.md` 作为持久指令文件，且可通过 `@路径` 语法导入其他文档¹。每次启动会话时，Claude 会将所有适用的 `CLAUDE.md` 文件（包括用户级和项目级）读取进上下文⁹¹⁰。`CLAUDE.md` 主要用于指示项目规范、架构决策、代码风格等（类似于 Codex 的 `AGENTS.md`）¹。组织级或用户级也可配置全局 `CLAUDE.md`，而子目录下的同名文件会在访问到相应路径下代码时加载（通过路径作用范围配置）¹¹¹²。
- **会话(Session)：** Claude Code 将每次交互视作一个会话，会话内容持续保存在本地，可通过 CLI 的 `--resume`、`--continue`、`--from-pr` 等参数或 `/resume` 命令恢复¹³¹⁴。每个会话启动时，都会重新加载上下文（来自 `CLAUDE.md`、之前会话的记忆等），但内部对话历史不自动跨会话继承（即新会话是新的空窗口，只携带 `CLAUDE.md` 和内存摘要）。会话数据保存在磁盘，可按需分支（branch）、

重命名、导出或与远端PR关联^{13 15}，支持跨终端和桌面环境继续同一工作流（通过 `--teleport`、桌面/移动App 等）。

- **自动记忆(Auto Memory)**: Claude Code 拥有“自动记忆”功能：它在 `~/.claude/projects/<project>/memory/` 目录下为每个项目维护记忆文件集合（首要文件为 `MEMORY.md`），记录构建命令、调试经验、偏好设置等信息^{16 17}。会话期间，Claude 可主动写入这些文件（界面上显示“Writing memory”）并在后续会话开始时读取摘要（前 200 行或 25KB 的 `MEMORY.md` 被加载作上下文）¹⁸。用户也可手动运行 `/memory` 命令查看、启用/禁用自动记忆或编辑记忆文件^{19 20}。需要注意的是，自动记忆是机器本地存储，只在同一设备和同一仓库范围内共享，不会同步到云端或其他设备^{21 22}。
- **Hooks / 权限**: Claude Code 允许用户在配置中定义生命周期钩子(Hooks)和权限设置。钩子可以在特定事件（如每次提交前、文件编辑后）触发外部命令，执行自动格式化或测试等^{1 23}。系统级指令也可通过 `--append-system-prompt` 参数在每次运行时强制附加。权限方面，Claude Code 提供完善的权限配置（如 `permissions.deny`、沙箱模式、环境变量限制等），可以禁止或放行特定工具、命令或文件路径²⁴。与 `CLAUDE.md` 中的建议不同，权限和沙箱设置是客户端强制执行的安全层，而 `CLAUDE.md` 指令本身则更像建议指南²⁴。因此 Claude Code 能读取、更新项目内的规则和记忆文件，但通过配置可以防止其违规操作（例如禁止编辑某些敏感目录或关闭沙箱）。
- **文件更新和安全边界**: Claude Code CLI 也支持编辑文件和运行命令（类似 Codex）。它可输出补丁并应用到项目文件。运行时会尊重 `CLAUDE.md` 指令，但遵循的是“尽力而为”原则，若指令含糊或冲突可能不会严格遵守^{25 23}。安全方面，用户可以通过权限配置禁止模型运行未经批准的命令（如执行危险脚本），并在每次操作前显示安全警告供用户确认²⁴。此外，自动记忆写入的文件都是纯文本，用户可以审查并随时删除。总之，Claude Code 在文件操作能力上与本地编码代理相似，但在安全与权限上提供了更多配置机制以防止越权编辑。

Cursor 能力边界

- **规则管理**: Cursor 使用专门的规则系统。项目级规则文件通常放在 `.cursor/rules/` 目录下（或者旧版的 `.cursorsrules` 文件），支持 `.mdc` 格式，可在头部通过 YAML 定义激活模式（Always、Auto Attached、Agent Requested、Manual 等）³。这些规则文件在编辑匹配的路径时被加载，为 AI 提供持续上下文。此外，Cursor 也兼容 `AGENTS.md` 文件（由 Agentic AI Foundation 推广），可读取通用指令³。因此可以将共用指令放在 `AGENTS.md`，项目专属规则仍放在 `.cursor/rules/`。多级规则支持使得 Cursor 的上下文指导更细粒度。
- **Notepads (记事板)**: Cursor 还提供「记事板 (Notepads)」功能（类似本地记忆）：用户可以创建富文本 Markdown 笔记，包含规则、链接、文件片段等，记事板通过 `@NotepadName` 在会话中引用²⁶²⁷。记事板会在本机自动同步到所有 Cursor 会话，充当“快捷上下文”。但它们并不纳入 Git 管理（存储在本地 SQLite 数据库中），且目前计划于 2025 年后弃用（建议对重要信息以代码文件形式版本管理）²⁸²⁹。总的来说，Cursor 通过规则文件和记事板为会话提供项目上下文，但没有像 Claude 那样的自动记忆数据库；记事板更像是一种用户手动维护的上下文片段。
- **Agent 模式**: Cursor 提供「Agent 模式」可以发起多步自动任务，在这种模式下也会尊重规则和记事板内容^{30 31}。它可以并行启动多个子任务（subagents）以处理不同子任务，并将结果合并。在 Agent 模式下，所有规则和记事板仍会作为上下文传递。不过，目前 Cursor 官方文档中并没有记载专门的“内置记忆”功能；其持久上下文主要依赖上述规则/记事板，而非自动总结的记忆文件。
- **其他**: Cursor 也能执行文件操作、运行本地命令和集成开发环境操作（如 Git diff）。它没有内置的 Git 提交功能，一般通过提醒用户或输出补丁由用户应用来更新文件。根据社区经验，Cursor 可与 GitHub Copilot Connector、Substack 等集成进行 PR 操作，但细节取决于具体插件/工作流。总之，Cursor 擅长利用规则文件和记事板提供上下文，但自动跨会话记忆较弱，项目多机器同步有时需要额外机制。

本地 Agent 相比纯对话工具的优势

本地编码 Agent 的最大优势在于对开发环境的原生访问能力：它们可以“看”到你的完整代码仓库、读写文件系统、运行测试和 CLI 工具【3†L19-L27】⁴。这意味着所有长期记忆都可以直接文件化并由版本控制系统管理。例如，项目层面的 `AGENTS.md`、`CLAUDE.md`、架构决策文档（ADR）、待办列表或记忆分类账等都放在仓库里，Agent 会自动加载并更新它们。与 ChatGPT 网页版相比，Agent 无需反复描述上下文，能够即时检索和修改项目状态。此外，本地 Agent 可以并行运行多个子任务，并且可以根据不同需求启用或禁用「记忆注入」，灵活度更高³²⁹。因为它们与 Git 紧密集成，每次修改都有差异（diff）记录，使审计和回滚更简单。总结来说，本地 Agent 在可控性、可审计性和与开发流程的一体化程度上优于纯对话接口，尤其适合需要长期追踪项目状态的情境。

本地 Agent 的风险

- **误改文件：**由于 Agent 拥有完整的文件系统读写权限，它可能会意外修改或删除长期记忆文件（规则、手动记录的状态等），尤其当规则或指令不够明确时。如果自动审批策略设置过宽松，Agent 可能进行未经充分验证的更改，因此需要谨慎控制“全自动”模式。建议对关键文件开启人工审核或只读模式。
- **规则冲突：**多个规则文件（例如全局 `AGENTS.md`、项目 `CLAUDE.md`、本地 `.claude/settings.json` 中的排除规则等）可能互相冲突，导致 Agent 行为不确定²⁴²³。例如，如果不同指令文件给出了相互矛盾的要求，Agent 可能会随意选取其中之一或产生不可预测的行为。需要通过设计清晰的层级结构和 `.claude/settings.local.json` 排除不相关文件来避免冲突³³。
- **本地记忆不跨设备：**Claude Code 的自动记忆和 Cursor 的记事板都只存储在本机，不会同步到云端或其他机器¹⁷²⁹。这意味着开发者如果在多台机器上协作，需要额外同步记忆文件，否则每台设备的 Agent 会像“重置”一样。虽然可以将重要信息版控或使用 MCP 服务解决，但这是设计上的限制。
- **权限过大：**默认情况下，本地 Agent 可以访问网络、运行命令、查看敏感文件。若权限管理不当，存在信息泄露或执行危险操作的风险。需要通过 Agent 的权限配置（如 Codex 的审批策略、Claude 的 `permissions.deny`、沙箱隔离等）来限制其可执行的操作范围，防止对关键资源（如密钥、生产数据库）造成影响。
- **记忆失真：**无论是 `CLAUDE.md` 还是自动记忆，都可能出现过时或错误的信息，误导 Agent。记忆条目如果过于模糊或超出上下文长度，也可能被 Agent 忽略或误用²⁵²³。因此需要定期审查、更新这些记忆文件，避免“上下文腐烂”。

推荐的高可信实践

- **使用 Git 管理记忆：**将所有长期指令文件（如 `AGENTS.md`、`CLAUDE.md`、`.cursor` 规则等）纳入 Git 仓库进行版本控制²⁸³。这样可以确保任何修改都可溯源、可审计，并能方便地回退或合并。Git 提交记录提供了完整的历史，也方便团队同步指令。
- **Patch / diff 审查：**要求 Agent 修改记忆文件时只输出补丁（或使用内置 `/diff` 命令查看变更），让开发者先行复查再提交⁶⁵。例如 Codex CLI 的 `/diff` 功能可展示所有改动，用户确认后再执行 `git commit`。这样能防止 Agent 直接对关键文件进行不必要的更改。
- **Memory 更新需人工确认：**对于自动生成的“记忆更新”，最好要求人工确认后再写入。例如，Claude Code 的 `/memory` 界面虽然允许自动记忆写入，但团队应定期检查 `~/.claude/projects/.../memory/` 文件，对生成的内容进行审查或整理，防止错误信息累积。不要让 Agent 在未经审阅的情况下自由改写重要的长期记忆文件。

- **受保护目录：** 将某些记忆文件或敏感文件放在受保护的目录下，限制 Agent 默认访问。例如，可以在权限配置中禁止 Agent 写入 `.claude/settings.*` 或 `.cursor/` 之外的目录，只允许特定工具或手动修改。对生产配置或关键文档采用只读策略，只允许人为审查后才变更。
- **规则文件版本化：** 制定统一的规则管理方案（如统一维护 AGENTS.md，并由各工具引用），并定期合并团队反馈。借鉴跨工具规范（如使用 AGENTS.md 作为“单一事实源”³），减少重复内容，便于维护。对项目规范，采用明确、简短且测试过的指令内容，避免在规则文件中堆砌过多冗余信息。³ ²⁸ 指出，将规则拆分并适当引用外部文件有助于保持规则文件精简并易于管理。

不确定项声明： 以上结论中，Codex CLI 的能力基于官方文档与社区反馈，其自动提交补丁和 diff 功能已明确⁶⁵；Claude Code 的记忆和会话特性有官方文档支持¹¹⁸；Cursor 的规则和 Notepad 功能基于官方指南及社区经验³⁴²⁶。若某些能力尚无官方说明（如 Cursor 未来是否会增加自动记忆），则按现有资料给出推测并已标注不确定。所有观点均以公开文档和专家撰稿为依据。

参考资料： OpenAI Codex 文档【3†L19-L27】⁶；Anthropic Claude Code 文档¹¹⁸；Cursor 官方文档与社区资源³⁴²⁶；开发者经验与最佳实践分析⁵³。

¹ Overview - Claude Code Docs

<https://code.claude.com/docs/en/overview>

² ⁹ ¹⁰ ¹¹ ¹² ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²⁰ ²¹ ²² ²³ ²⁴ ²⁵ ³³ How Claude remembers your project - Claude Code Docs

<https://code.claude.com/docs/en/memory>

³ Complete Guide to CLAUDE.md and AGENTS.md 2026 | Data Science Collective

<https://medium.com/data-science-collective/the-complete-guide-to-ai-agent-memory-files-claude-md-agents-md-and-beyond-49ea0df5c5a9>

⁴ OpenAI Codex CLI, how does it work?

<https://www.philschmid.de/openai-codex-cli>

⁵ Exploring the OpenAI Codex CLI Source Code

<https://zenn.dev/takiko/articles/e2b8065158c8d0?locale=en>

⁶ ³² Slash commands in Codex CLI | OpenAI Developers

<https://developers.openai.com/codex/cli/slash-commands>

⁷ ⁸ PRs opened by Codex are not updated with latest changes - Codex - OpenAI Developer Community

<https://community.openai.com/t/prs-opened-by-codex-are-not-updated-with-latest-changes/1266174>

¹³ ¹⁴ ¹⁵ Manage sessions - Claude Code Docs

<https://code.claude.com/docs/en/sessions>

²⁶ ²⁷ ²⁸ ²⁹ ³⁰ ³¹ Using Cursor Notepads for Context Management | Developing with AI Tools | Steve Kinney

<https://stevekinney.com/courses/ai-development/cursor-notepads>

³⁴ Cursor Rules Developer Guide — NVIDIA NeMo Agent Toolkit (1.2)

<https://docs.nvidia.com/nemo/agent-toolkit/1.2/extend/cursor-rules-developer-guide.html>